机器数的表示与运算

1. 机器数

- 计算机中的数称为机器数
- 构成:
  - 符号位 + 真值

“0” → 表示正

“1” → 表示负

[例]

+52 = +0110100 = 0 0110100

符号位　真值

-52 = -0110100 = 1 0110100

符号位　真值

2. 机器数的表示

- 机器数的表示方法:
  - 原码
  - 反码
  - 补码

原码

- 最高位为符号位，其余为真值部分。
  - “0”表示正
  - “1”表示负
- 优点：
  - 真值和其原码表示之间的对应关系简单，容易理解;
- 缺点：
  - 计算机中用原码进行加减运算比较困难
  - 0的表示不唯一。

数0的原码

- 8位数0的原码:
  - +0=0 0000000
  - -0=1 0000000

即：数0的原码不唯一。

反码

对一个机器数X:

- 若X>0
  - $[X]_{反}=[X]_{原}$
- 若X<0
  - $[X]_{反}$= 对应原码的符号位不变，数值部分按位求反

[例]

- X= -52 = -0110100

  $[X]_{原}$=1 0110100

  $[X]_{反}$=1 1001011

0的反码：

$[+0]_{反}=00000000$

$[-0]_{反}=11111111$

即：数0的反码也不是唯一的。

补码

定义：

- 若X>0
  - $[X]_{补} = [X]_{反} = [X]_{原}$
- 若X<0
  - $[X]_{补} = [X]_{反} + 1$

[例]

- X=-52= -0110100
  - $[X]_{原}$=10110100
  - $[X]_{反}$=11001011
  - $[X]_{补}$= $[X]_{反}$+1=11001100

0的补码：

- $[+0]_{补} = [+0]_{原} = 00000000$
- $[-0]_{补} = [-0]_{反} + 1 = 11111111 + 1$

$= \underline{1}\ 00000000$

对8位字长，进位被舍掉

现代计算机中多采用补码

3. 补码数的运算

- 通过引进补码，可将减法运算转换为加法运算。
- 因为:
  - $[X+Y]_{补}=[X]_{补}+[Y]_{补}$
  - $[X-Y]_{补}=[X+(-Y)]_{补}$
  - $=[X]_{补}+[-Y]_{补}$

例1：

- 66-51=66+（-51）=15
- 用二进制补码运算:
  - $[+66]_{补}=[+66]_{原}=01000010$
  - $[-51]_{原}=10110011$
  - $[-51]_{补}=11001101$
  - $[+66]_{补}+[-51]_{补}=\boxed{1}00001111$

    $=15$

例2：

- X=-52=-0110100，Y=116=+1110100，求X+Y=？
  - $[X]_{原}$=10110100
  - $[X]_{补}$= $[X]_{反}$+1=11001100
  - $[Y]_{补}$= $[Y]_{原}$=01110100
  - $[X+Y]_{补}$= $[X]_{补}$+ $[Y]_{补}$

    =11001100+01110100

    =01000000
  - X+Y=+1000000

- 现代计算机系统中，程序设计时，负数可用“-”表示，由编译系统将其转换为补码。
- 例：
  - 若输入数=-3
  - 程序编译后的值=FDH